

Лабораторная работа №2

Ромицына Анастасия Романовна

18 декабря 2025

Российский университет дружбы народов

Основная цель работы — изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

Выполнение лабораторной работы

Введем задание из лабораторной работы “Кортежи”

```
[1]: ()  
[1]: ()  
[2]: favoritelang = ("Python", "Julia", "R")  
[2]: ("Python", "Julia", "R")  
[3]: x1 = (1, 2, 3)  
[3]: (1, 2, 3)  
[4]: x2 = (1, 2.0, "tmp")  
[4]: (1, 2.0, "tmp")  
[5]: x3 = (a=2, b=1+2)  
[5]: (a = 2, b = 3)  
[6]: length(x2)  
[6]: 3  
[7]: x2[1], x2[2], x2[3]  
[7]: (1, 2.0, "tmp")  
[8]: c = x1[2] + x1[3]  
[8]: 5  
[9]: x3.a, x3.b, x3[2]  
[9]: (2, 3, 3)  
[10]: in("tmp", x2), 0 in x2  
[10]: (true, false)  
[11]: phonebook = Dict{"Иванов И.И." => ("867-5309", "333-5544"), "Бухгалтерия" => "555-2368"}  
[11]: Dict{String, Any} with 2 entries:  
      "Бухгалтерия" => "555-2368"  
      "Иванов И.И." => ("867-5309", "333-5544")
```

Рис. 1: Кортежи

Введем задание из лабораторной работы “Словари”

```
[12]: keys(phonebook)
[12]: KeySet for a Dict(String, Any) with 2 entries. Keys:
      "Бухартегин"
      "Иванов И.И."
[13]: values(phonebook)
[13]: ValueIterator for a Dict(String, Any) with 2 entries. Values:
      "555-2368"
      ("867-5309", "333-5544")
[14]: pairs(phonebook)
[14]: Dict{String, Any} with 2 entries:
      "Бухартегин" => "555-2368"
      "Иванов И.И." => ("867-5309", "333-5544")
[15]: haskey(phonebook, "Иванов И.И.")
[15]: true
[16]: phonebook["Сидоров П.С."] = "555-3344"
[16]: "555-3344"
[17]: pop!(phonebook, "Иванов И.И.")
```

Рис. 2: Словари

Введем задание из лабораторной работы “Множества”

```
[1]: [len(string.ascii_lowercase) * 10, 10, 'Yes' * 10, 'Yes' * 10, 'Yes' * 10, 'Yes' * 10, 'Yes' * 10, 'Yes' * 10, 'Yes' * 10, 'Yes' * 10]
[1]: A = Set([1, 2, 3, 4, 5])
[1]: Set([1000]) with 1 elements:
1000
[1]: B = Set(["microbiology"])
[1]: Set([1000]) with 1 elements:
1000
[1]: S1 = Set([1, 2, 3])
S2 = Set([1, 2, 3])
Intersection(S1, S2)
S3 = Set([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
S4 = Set([1, 2, 3, 4])
Intersection(S3, S4)
[1]: True
[1]: Complement(S1, S2)
[1]: Set([1000]) with 1 elements:
1000
[1]: B = Set([1000, 1001])
[1]: Set([1000]) with 1 elements:
1000
[1]: B = Set([0, 1, 2, 3])
[1]: Set([1000]) with 1 elements:
1000
[1]: Intersection(S3, S4)
```

Рис. 3: Множества

Введем задание из лабораторной работы “Массивы”

```
[27]: push!(S4, 99)
[27]: Set{Int64} with 4 elements:
      2
      99
      3
      1
[28]: pop!(S4)
[28]: 2
[29]: empty_array_1 = []
[29]: Any[]
[30]: empty_array_2 = {Int64}[]
      empty_array_3 = {Float64}[]
[30]: Float64[]
[31]: a = [1, 2, 3]
[31]: 3-element Vector{Int64}:
      1
      2
      3
[32]: b = [1 2 3]
[32]: 1x3 Matrix{Int64}:
      1  2  3
[33]: A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]]
      B = [[1 2 3]; [4 5 6]; [7 8 9]]
[33]: 3x3 Matrix{Int64}:
      1  2  3
      4  5  6
      7  8  9
[34]: c = rand(1,8)
[34]: 1x8 Matrix{Float64}:
      0.0858007  0.219772  0.147176  0.542635  - 0.0313715  0.38277  0.0495543
```

Рис. 4: Массивы

Введем задание из лабораторной работы “Массивы”

```
[35]: C = rand(2,3);

[36]: D = rand(4, 3, 2)

[36]: 4x3x2 Array{Float64, 3}:
      [1, 1, 1] =
      0.978744  0.793065  0.370237
      0.617183  0.840505  0.301234
      0.20504  0.601418  0.0332036
      0.700504  0.0223628  0.19859

      [1, 1, 2] =
      0.942642  0.010429  0.303208
      0.55964  0.406944  0.23279
      0.10795  0.743071  0.636717
      0.788783  0.165509  0.882029

[37]: roots = [sqrt(i) for i in 1:10]

[37]: 10-element Vector{Float64}:
 1.0
 1.4142135623730951
 1.7320508075688772
 2.0
 2.23606797749979
 2.449489742783178
 2.6457513110645907
 2.8284271247461903
 3.0
 3.1622776601683795

[38]: ar_1 = [3^i*i^2 for i in 1:2:9]

[38]: 5-element Vector{Int64}:
 3
 27
 75
 147
 243

[39]: ar_2=[i^2 for i=1:10 if (i^2%5!=0 && i^2%4!=0)]

[39]: 4-element Vector{Int64}:
 1
 9
 49
 81
```

Рис. 5: Массивы

Введем задание из лабораторной работы “Массивы”

```
[40]: ones(5)

[40]: 5-element Vector{Float64}:
 1.0
 1.0
 1.0
 1.0
 1.0

[41]: ones(2,3)

[41]: 2x3 Matrix{Float64}:
 1.0  1.0  1.0
 1.0  1.0  1.0

[42]: zeros(4)

[42]: 4-element Vector{Float64}:
 0.0
 0.0
 0.0
 0.0

[43]: fill(3.5,(3,2))

[43]: 3x2 Matrix{Float64}:
 3.5  3.5
 3.5  3.5
 3.5  3.5

[44]: repeat([1,2],3,3)
      repeat([1 2],3,3)

[44]: 3x6 Matrix{Int64}:
 1  2  1  2  1  2
 1  2  1  2  1  2
 1  2  1  2  1  2

[45]: a = collect(1:12)
      b = reshape(a,(2,6))

[45]: 2x6 Matrix{Int64}:
 1  3  5  7  9  11
 2  4  6  8  10  12
```

Рис. 6: Массивы

Введем задание из лабораторной работы “Массивы”

```
[46]: b'
```

```
[46]: 6x2 adjoint(::Matrix{Int64}) with eltype Int64:
      1  2
      3  4
      5  6
      7  8
      9 10
     11 12
```

```
[47]: c = transpose(b)
```

```
[47]: 6x2 transpose(::Matrix{Int64}) with eltype Int64:
      1  2
      3  4
      5  6
      7  8
      9 10
     11 12
```

```
[48]: ar = rand(10:20, 10, 5)
```

```
[48]: 10x5 Matrix{Int64}:
      10 19 12 19 16
      14 13 10 10 19
      18 19 16 13 10
      11 12 20 16 11
      20 11 16 12 17
      12 15 11 19 12
      15 10 17 13 10
      16 19 13 12 10
      14 14 14 12 18
      12 17 20 15 12
```

```
[49]: ar[:, 2]
```

```
[49]: 10-element Vector{Int64}:
      19
      13
      19
      12
      11
      15
      10
      19
      14
      17
```

Рис. 7: Массивы

Введем задание из лабораторной работы “Массивы”

```
[50]: ar[:, [2, 5]]
```

```
[50]: 10x2 Matrix{Int64}:  
 19 16  
 13 19  
 19 10  
 12 11  
 11 17  
 15 12  
 10 10  
 19 10  
 14 18  
 17 12
```

```
[51]: ar[:, 2:4]
```

```
[51]: 10x3 Matrix{Int64}:  
 19 12 19  
 13 10 10  
 19 16 13  
 12 20 16  
 11 16 12  
 15 11 19  
 10 17 13  
 19 13 12  
 14 14 12  
 17 20 15
```

```
[52]: ar[[2, 4, 6], [1, 5]]
```

```
[52]: 3x2 Matrix{Int64}:  
 14 19  
 11 11  
 12 12
```

```
[53]: ar[1, 3:end]
```

```
[53]: 3-element Vector{Int64}:  
 12  
 19  
 16
```

Рис. 8: Массивы

Введем задание из лабораторной работы “Массивы”

```
[54]: sort(ar,dims=1)

[54]: 10x5 Matrix{Int64}:
 10 10 10 10 10
 11 11 11 12 10
 12 12 12 12 10
 12 13 13 12 11
 14 14 14 13 12
 14 15 16 13 12
 15 17 16 15 16
 16 19 17 16 17
 18 19 20 19 18
 20 19 20 19 19

[55]: sort(ar,dims=2)

[55]: 10x5 Matrix{Int64}:
 10 12 16 19 19
 10 10 13 14 19
 10 13 16 18 19
 11 11 12 16 20
 11 12 16 17 20
 11 12 12 15 19
 10 10 13 15 17
 10 12 13 16 19
 12 14 14 14 18
 12 12 15 17 20

[56]: ar .> 14

[56]: 10x5 BitMatrix:
 0 1 0 1 1
 0 0 0 0 1
 1 1 1 0 0
 0 0 1 1 0
 1 0 1 0 1
 0 1 0 1 0
 1 0 1 0 0
 1 1 0 0 0
 0 0 0 0 1
 0 1 1 1 0
```

Рис. 9: Массивы

Задание 1,2

```
[57]: findall(ar -> 14)

[57]: 22-element Vector{CartesianIndex{2}}:
 CartesianIndex{2, 1}
 CartesianIndex{5, 1}
 CartesianIndex{7, 1}
 CartesianIndex{8, 1}
 CartesianIndex{1, 2}
 CartesianIndex{3, 2}
 CartesianIndex{6, 2}
 CartesianIndex{8, 2}
 CartesianIndex{10, 2}
 CartesianIndex{2, 3}
 CartesianIndex{4, 3}
 CartesianIndex{5, 3}
 CartesianIndex{7, 3}
 CartesianIndex{10, 3}
 CartesianIndex{1, 4}
 CartesianIndex{4, 4}
 CartesianIndex{6, 4}
 CartesianIndex{10, 4}
 CartesianIndex{1, 5}
 CartesianIndex{2, 5}
 CartesianIndex{5, 5}
 CartesianIndex{9, 5}

[58]: A = Set{0, 3, 4, 9}
      B = Set{1, 5, 4, 7}
      C = Set{0, 1, 4, 7, 8, 9}

      P = union(intersect(A, B), intersect(A, C), intersect(B, C))
      println(P)

      Set{0, 4, 7, 9, 5, 1}

[59]: set1 = Set{"apple", "orange", "banana"}
      set2 = Set{"banana", "melon", "cherry"}
      intersection = intersect(set1, set2)
      println(intersection)

      Set{"banana"}

[60]: set3 = Set{10, 20, 30}
      set4 = Set{20, 40, 50}
      difference = setdiff(set3, set4)
      println(difference)

      Set{10, 30}
```

Рис. 10: Решим задание 1 и 2 из лабораторной работы ☒ = ☒☒☒☒☒☒☒☒☒☒☒☒☒☒☒☒☒.

Задание 3.1-3.9

```

8  # 2D
9  vector<vector<int>>
10  graph( int n, vector<vector<int>> E, int s, int t, int k) {
11      vector<vector<int>> G(n, vector<int>(n));
12      for (int i = 0; i < E.size(); i++)
13          for (int j = 0; j < E[i].size(); j++)
14              G[E[i][j]][E[j][i]] = 1;
15      vector<vector<double>> dist(n, vector<double>(n, 1E5));
16      priority_queue<int> pq;
17      dist[s][s] = 0;
18      pq.push(s);
19      while (!pq.empty()) {
20          int u = pq.top(); pq.pop();
21          for (int v = 0; v < n; v++)
22              if (G[u][v] && dist[u][u] + 1 < dist[u][v]) {
23                  dist[u][v] = dist[u][u] + 1;
24                  pq.push(v);
25              }
26      }
27      return dist[t][t];
28  }
29  int n;
30  vector<vector<int>> G;
31  priority_queue<int> pq;
32  vector<vector<double>> dist;
33  int s, t, k;
34  vector<vector<int>> E;
35  priority_queue<int> pq2;
36  vector<vector<double>> dist2;
37  int u, v;
38  vector<vector<int>> G2;
39  priority_queue<int> pq3;
40  vector<vector<double>> dist3;
41  int u2, v2;
42  vector<vector<int>> G3;
43  priority_queue<int> pq4;
44  vector<vector<double>> dist4;
45  int u3, v3;
46  vector<vector<int>> G4;
47  priority_queue<int> pq5;
48  vector<vector<double>> dist5;
49  int u4, v4;
50  vector<vector<int>> G5;
51  priority_queue<int> pq6;
52  vector<vector<double>> dist6;
53  int u5, v5;
54  vector<vector<int>> G6;
55  priority_queue<int> pq7;
56  vector<vector<double>> dist7;
57  int u6, v6;
58  vector<vector<int>> G7;
59  priority_queue<int> pq8;
60  vector<vector<double>> dist8;
61  int u7, v7;
62  vector<vector<int>> G8;
63  priority_queue<int> pq9;
64  vector<vector<double>> dist9;
65  int u8, v8;
66  vector<vector<int>> G9;
67  priority_queue<int> pq10;
68  vector<vector<double>> dist10;
69  int u9, v9;
70  vector<vector<int>> G10;
71  priority_queue<int> pq11;
72  vector<vector<double>> dist11;
73  int u10, v10;
74  vector<vector<int>> G11;
75  priority_queue<int> pq12;
76  vector<vector<double>> dist12;
77  int u11, v11;
78  vector<vector<int>> G12;
79  priority_queue<int> pq13;
80  vector<vector<double>> dist13;
81  int u12, v12;
82  vector<vector<int>> G13;
83  priority_queue<int> pq14;
84  vector<vector<double>> dist14;
85  int u13, v13;
86  vector<vector<int>> G14;
87  priority_queue<int> pq15;
88  vector<vector<double>> dist15;
89  int u14, v14;
90  vector<vector<int>> G15;
91  priority_queue<int> pq16;
92  vector<vector<double>> dist16;
93  int u15, v15;
94  vector<vector<int>> G16;
95  priority_queue<int> pq17;
96  vector<vector<double>> dist17;
97  int u16, v16;
98  vector<vector<int>> G17;
99  priority_queue<int> pq18;
100 vector<vector<double>> dist18;
101 int u17, v17;
102 vector<vector<int>> G18;
103 priority_queue<int> pq19;
104 vector<vector<double>> dist19;
105 int u18, v18;
106 vector<vector<int>> G19;
107 priority_queue<int> pq20;
108 vector<vector<double>> dist20;
109 int u19, v19;
110 vector<vector<int>> G20;
111 priority_queue<int> pq21;
112 vector<vector<double>> dist21;
113 int u20, v20;
114 vector<vector<int>> G21;
115 priority_queue<int> pq22;
116 vector<vector<double>> dist22;
117 int u21, v21;
118 vector<vector<int>> G22;
119 priority_queue<int> pq23;
120 vector<vector<double>> dist23;
121 int u22, v22;
122 vector<vector<int>> G23;
123 priority_queue<int> pq24;
124 vector<vector<double>> dist24;
125 int u23, v23;
126 vector<vector<int>> G24;
127 priority_queue<int> pq25;
128 vector<vector<double>> dist25;
129 int u24, v24;
130 vector<vector<int>> G25;
131 priority_queue<int> pq26;
132 vector<vector<double>> dist26;
133 int u25, v25;
134 vector<vector<int>> G26;
135 priority_queue<int> pq27;
136 vector<vector<double>> dist27;
137 int u26, v26;
138 vector<vector<int>> G27;
139 priority_queue<int> pq28;
140 vector<vector<double>> dist28;
141 int u27, v27;
142 vector<vector<int>> G28;
143 priority_queue<int> pq29;
144 vector<vector<double>> dist29;
145 int u28, v28;
146 vector<vector<int>> G29;
147 priority_queue<int> pq30;
148 vector<vector<double>> dist30;
149 int u29, v29;
150 vector<vector<int>> G30;
151 priority_queue<int> pq31;
152 vector<vector<double>> dist31;
153 int u30, v30;
154 vector<vector<int>> G31;
155 priority_queue<int> pq32;
156 vector<vector<double>> dist32;
157 int u31, v31;
158 vector<vector<int>> G32;
159 priority_queue<int> pq33;
160 vector<vector<double>> dist33;
161 int u32, v32;
162 vector<vector<int>> G33;
163 priority_queue<int> pq34;
164 vector<vector<double>> dist34;
165 int u33, v33;
166 vector<vector<int>> G34;
167 priority_queue<int> pq35;
168 vector<vector<double>> dist35;
169 int u34, v34;
170 vector<vector<int>> G35;
171 priority_queue<int> pq36;
172 vector<vector<double>> dist36;
173 int u35, v35;
174 vector<vector<int>> G36;
175 priority_queue<int> pq37;
176 vector<vector<double>> dist37;
177 int u36, v36;
178 vector<vector<int>> G37;
179 priority_queue<int> pq38;
180 vector<vector<double>> dist38;
181 int u37, v37;
182 vector<vector<int>> G38;
183 priority_queue<int> pq39;
184 vector<vector<double>> dist39;
185 int u38, v38;
186 vector<vector<int>> G39;
187 priority_queue<int> pq40;
188 vector<vector<double>> dist40;
189 int u39, v39;
190 vector<vector<int>> G40;
191 priority_queue<int> pq41;
192 vector<vector<double>> dist41;
193 int u40, v40;
194 vector<vector<int>> G41;
195 priority_queue<int> pq42;
196 vector<vector<double>> dist42;
197 int u41, v41;
198 vector<vector<int>> G42;
199 priority_queue<int> pq43;
200 vector<vector<double>> dist43;
201 int u42, v42;
202 vector<vector<int>> G43;
203 priority_queue<int> pq44;
204 vector<vector<double>> dist44;
205 int u43, v43;
206 vector<vector<int>> G44;
207 priority_queue<int> pq45;
208 vector<vector<double>> dist45;
209 int u44, v44;
210 vector<vector<int>> G45;
211 priority_queue<int> pq46;
212 vector<vector<double>> dist46;
213 int u45, v45;
214 vector<vector<int>> G46;
215 priority_queue<int> pq47;
216 vector<vector<double>> dist47;
217 int u46, v46;
218 vector<vector<int>> G47;
219 priority_queue<int> pq48;
220 vector<vector<double>> dist48;
221 int u47, v47;
222 vector<vector<int>> G48;
223 priority_queue<int> pq49;
224 vector<vector<double>> dist49;
225 int u48, v48;
226 vector<vector<int>> G49;
227 priority_queue<int> pq50;
228 vector<vector<double>> dist50;
229 int u49, v49;
230 vector<vector<int>> G50;
231 priority_queue<int> pq51;
232 vector<vector<double>> dist51;
233 int u50, v50;
234 vector<vector<int>> G51;
235 priority_queue<int> pq52;
236 vector<vector<double>> dist52;
237 int u51, v51;
238 vector<vector<int>> G52;
239 priority_queue<int> pq53;
240 vector<vector<double>> dist53;
241 int u52, v52;
242 vector<vector<int>> G53;
243 priority_queue<int> pq54;
244 vector<vector<double>> dist54;
245 int u53, v53;
246 vector<vector<int>> G54;
247 priority_queue<int> pq55;
248 vector<vector<double>> dist55;
249 int u54, v54;
250 vector<vector<int>> G55;
251 priority_queue<int> pq56;
252 vector<vector<double>> dist56;
253 int u55, v55;
254 vector<vector<int>> G56;
255 priority_queue<int> pq57;
256 vector<vector<double>> dist57;
257 int u56, v56;
258 vector<vector<int>> G57;
259 priority_queue<int> pq58;
260 vector<vector<
```

Рис. 11: Решим задание 3.1-3.9 с разными видами массивов из лабораторной работы

Задание 3.10-3.13

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000
```

Рис. 12: Решим задание 3.10-3.13 с разными видами массивов из лабораторной работы

Задание 3.14

```
1 // Задание 3.14
2 // Решение с использованием массивов
3 //
4 // Ввод: строка, состоящая из цифр и пробелов.
5 // Вывод: строка, состоящая из цифр и пробелов.
6 //
7 // Пример:
8 // Ввод: 1 2 3 4 5 6 7 8 9 10
9 // Вывод: 1 2 3 4 5 6 7 8 9 10
10
11 // Решение:
12 // 1. Создаем массив для хранения строки.
13 // 2. Заполняем массив строкой.
14 // 3. Выводим массив.
15
16 #include <iostream>
17 #include <string>
18
19 using namespace std;
20
21 int main()
22 {
23     string s;
24     cin >> s;
25     cout << s << endl;
26     return 0;
27 }
```

Рис. 13: Решим задание 3.14 с разными видами массивов из лабораторной работы



```
1 // Задание 4. Решим задание 4 с разными видами массивов из лабораторной работы
2 #include <iostream>
3 using namespace std;
4
5 // Функция для подсчета суммы элементов массива
6 int sum(int arr[], int n) {
7     int s = 0;
8     for (int i = 0; i < n; i++) {
9         s += arr[i];
10    }
11    return s;
12 }
13
14 // Функция для подсчета суммы элементов массива
15 double sum_double(double arr[], int n) {
16     double s = 0;
17     for (int i = 0; i < n; i++) {
18         s += arr[i];
19     }
20     return s;
21 }
22
23 // Функция для подсчета суммы элементов массива
24 string sum_string(string arr[], int n) {
25     string s = "";
26     for (int i = 0; i < n; i++) {
27         s += arr[i] + " ";
28     }
29     return s;
30 }
31
32 int main() {
33     int n;
34     cout << "Введите количество элементов массива: ";
35     cin >> n;
36
37     // Массив целых чисел
38     int arr1[n];
39     for (int i = 0; i < n; i++) {
40         arr1[i] = i + 1;
41     }
42     cout << "Сумма элементов массива: " << sum(arr1, n) << endl;
43
44     // Массив вещественных чисел
45     double arr2[n];
46     for (int i = 0; i < n; i++) {
47         arr2[i] = i + 1.5;
48     }
49     cout << "Сумма элементов массива: " << sum_double(arr2, n) << endl;
50
51     // Массив строк
52     string arr3[n];
53     for (int i = 0; i < n; i++) {
54         arr3[i] = "Hello World!";
55     }
56     cout << "Сумма элементов массива: " << sum_string(arr3, n) << endl;
57 }
```

Рис. 14: Решим задание 4 с разными видами массивов из лабораторной работы

Задание 5 и 6

[illegible]

Рис. 15: Решим задание 5 и 6 с разными видами массивов из лабораторной работы

Мы смогли изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.